



FUNDAMENTOS Y CONFIGURACIÓN DEL ENTORNO

En este primer capítulo, se asientan las bases para emprender un viaje hacia el desarrollo de aplicaciones móviles, estableciendo inicialmente los principios de Flutter y Dart; para luego, trabajar en una guía paso a paso que permita la configuración del entorno de desarrollo, asegurando de esta manera la preparación para comenzar a codificar sin contratiempos.

Al concluir este capítulo, se pondrá en práctica lo aprendido, desarrollando y ejecutando una primera aplicación Flutter, este ejercicio introductorio no solo reforzará la comprensión de los conceptos básicos, sino que también proporcionará una sensación tangible de logro al ver la creación en acción.

1.1 Introducción

Conforme los desarrolladores se familiarizaron con el entorno móvil, adoptaron estructuras de diseño y marcos de trabajo similares a los creados para abordar el entorno web. Y de manera análoga a las dificultades presentadas en el desarrollo web debido a la diversidad de navegadores, en el ámbito móvil se han enfrentado al desafío de crear código compatible con dispositivos iOS y Android, esto estableció la aspiración de contar con una base de código que funcione de manera fluida en ambas plataformas.

Flutter representa un avance significativo en cuanto a la aspiración mencionada, ya que ha transformado la manera en que se desarrollan aplicaciones nativas para diversas plataformas. Su principal innovación radica en la posibilidad de utilizar un solo conjunto de código fuente, que luego puede compilarse para funcionar en diferentes plataformas.

1.1.1 ¿Qué es Flutter?

De manera precisa, Flutter es un kit de desarrollo de software (SDK, por sus siglas en inglés) que utiliza el lenguaje de programación Dart, optimizado para ofrecer un rendimiento eficiente en el entorno de Flutter.

Una característica muy relevante, como menciona Bailey & Biessek, p. (2021, p. 5) “Flutter, y el lenguaje de programación Dart del que depende, fueron creados por Google, y aunque son de código abierto, siguen contando con el respaldo de Google” [Traducción propia]. Esto es relevante porque proporciona una garantía de soporte y desarrollo continuo, ya que el respaldo de una empresa como Google puede inspirar confianza en los desarrolladores y las empresas que eligen utilizar estas tecnologías para sus proyectos. Además, el respaldo de Google suele estar asociado con una mayor estabilidad, recursos y una comunidad de desarrollo activa, lo que es notable para el éxito y la longevidad de una tecnología en el panorama tecnológico actual.

Otra de las características más distintivas de Flutter es su motor de renderizado propio, basado en el lenguaje de programación Dart, esto permite compilar en código nativo, logrando un rendimiento comparable al de las aplicaciones nativas tradicionales.

Además, Flutter facilita el desarrollo rápido con características como el "hot reload", que permite ver los cambios en tiempo real sin necesidad de reiniciar la aplicación, mejorando significativamente tu productividad durante el desarrollo de la aplicación.

Flutter es un marco de desarrollo que ha experimentado una evolución constante desde su lanzamiento, con actualizaciones regulares que introducen nuevas características, mejoras de rendimiento y correcciones de errores. Cada versión trae consigo mejoras significativas, dirigidas tanto a ampliar las capacidades del framework como a mejorar la experiencia de desarrollo. A continuación, se presenta un breve resumen de algunas de las versiones más notables de Flutter a lo largo del tiempo:

- Flutter 1.0 (diciembre de 2018) fue la primera versión estable, marcando el punto de partida como un framework sólido para el desarrollo de aplicaciones móviles para Android e iOS.

- Flutter 2.0 (marzo de 2021) marcó un hito importante al introducir soporte para el desarrollo web, lo que permite usar el mismo código base para crear aplicaciones no solo para dispositivos móviles sino también para la web.
- Flutter 3.0 (mayo de 2022) introdujo el soporte para el desarrollo de aplicaciones para Windows, ampliando aún más la versatilidad de Flutter para abarcar todas las plataformas principales: móvil, web y escritorio.

1.1.2 Características principales de Flutter

A continuación, se presentan las características más sobresalientes de esta tecnología:

- **Desarrollo multiplataforma:** Durante años, se ha prometido la capacidad de escribir un código único capaz de ejecutarse en cualquier plataforma; Flutter podría considerarse como el intento más prometedor hasta la fecha para alcanzar dicho objetivo.
- **Hot Reload:** Una característica destacada, conocida como hot reload (recarga en caliente), habilita la actualización del código y la visualización inmediata de los cambios en tiempo real, sin requerir una recompilación o reinicio de la aplicación.
- **Open source:** Flutter es un proyecto de código abierto, lo que invita a la comunidad de desarrolladores a contribuir y enriquecer continuamente su marco de trabajo. Esta colaboración fomenta una evolución acelerada y facilita la integración de nuevas funcionalidades, beneficiando a todos los usuarios de la plataforma.
- **Lenguaje Dart:** Flutter se apoya en Dart como su lenguaje de programación. La transición a Dart no debería generar inquietud, ya que su similitud con Java, C#, y otros lenguajes influenciados por C, promete una curva de aprendizaje amigable y una experiencia de programación confortable para los desarrolladores ya instruidos en dichos lenguajes.
- **Conjunto de widgets:** Flutter ofrece una amplia gama de widgets preconstruidos que siguen las directrices de Material Design¹ y Cupertino², facilitando así la creación de interfaces hermosas y responsivas.
- **Rendimiento nativo:** Gracias a que Flutter emplea la compilación anticipada (AOT, por sus siglas en inglés: Ahead Of Time) para transformar el código Dart en código de máquina nativo. Esto significa que las aplicaciones se ejecutan directamente en el hardware del dispositivo, resultando en un rendimiento rápido y fluido, similar al de las aplicaciones nativas.

1.1.3 ¿Qué es Dart?

Como se ha señalado, el desarrollo de aplicaciones en Flutter requiere el uso de Dart, un lenguaje de programación creado por Google. Dart es un lenguaje de programación contemporáneo, hizo su debut el 14 de

¹ Material Design es una guía de estilo visual desarrollada por Google, que establece principios de diseño para una interfaz de usuario intuitiva y atractiva.

² Cupertino es un estilo de diseño desarrollado por Apple, caracterizado por su aspecto limpio y moderno, que se utiliza principalmente en el sistema operativo iOS.

noviembre de 2013, su segunda versión se lanzó en agosto de 2018, mientras que la versión más actual, 3.2.5, se renovó el 18 de enero de 2024. Esto demuestra el esfuerzo sostenido de Google por desarrollar y perfeccionar el lenguaje, lo que lo se puede considerar una excelente opción, dada la dedicación de la empresa al mantenimiento y soporte continuo de esta tecnología.

Una característica importante a la hora de elegir un lenguaje tiene que ver con la curva de aprendizaje, en ese sentido, como afirma Alessandria & Kayfitz, p. (2021, p. 41) “El lenguaje utiliza una sintaxis que será inmediatamente familiar para cualquier desarrollador que ya conozca un lenguaje de estilo C” [Traducción propia]. Lo que lo hace ideal para los programadores que tengan experiencia previa con el lenguaje C, ya que su manejo no presentará mayor dificultad, reduciendo así la curva de aprendizaje de esta tecnología. Otras características del lenguaje se señalan en el siguiente punto.

1.1.4 Características principales de Dart

Como se ha comprometido previamente, a continuación, se detallan las características que, desde una perspectiva objetiva, posicionan a Dart como un lenguaje digno de consideración, incluso independientemente de la existencia de Flutter.

- **Tiene un propósito general:** Puede ser usado para desarrollo móvil, para aplicaciones de escritorio o para otros tipos de desarrollo.
- **Orientado a objetos y fuertemente tipado:** Estas características permiten escribir código más estructurado y prevenir errores en tiempo de ejecución.
- **Soporte para programación reactiva:** Facilita la creación de aplicaciones altamente interactivas con características como streams y futures.

En general, comenzar a desarrollar con Flutter y Dart es iniciar un viaje hacia la creación de aplicaciones móviles hermosas, eficientes y multiplataforma.

1.2 Programación con el lenguaje Dart

El objetivo de esta sección es proporcionar una introducción al uso de Dart, de modo que al escribir la primera aplicación en Flutter, el enfoque pueda estar más en Flutter y no tanto en Dart. Si bien esta no es una guía completa, se espera que sea suficiente para facilitar un comienzo en el desarrollo con Dart, esto principalmente asumiendo que cuenta con experiencia previa en lenguajes de programación derivados del lenguaje C.

Para los ejemplos presentados en esta sección, se hará uso de DartPad³. Según Carew (2024) “DartPad es un editor en línea gratuito y de código abierto diseñado para ayudar a los desarrolladores a aprender sobre Dart y Flutter. Se puede acceder a él en dartpad.dev” [Traducción propia]. De manera precisa, DartPad representa la eliminación de instalaciones adicionales en la computadora y acceder de manera rápida a un editor donde se pueda practicar con el lenguaje Dart.

³ <https://dartpad.dev/>

1.2.1 Variantes del “Hola Mundo”

Al acceder a DartPad por primera vez, se observará una interfaz que se asemeja a la imagen proporcionada a continuación.

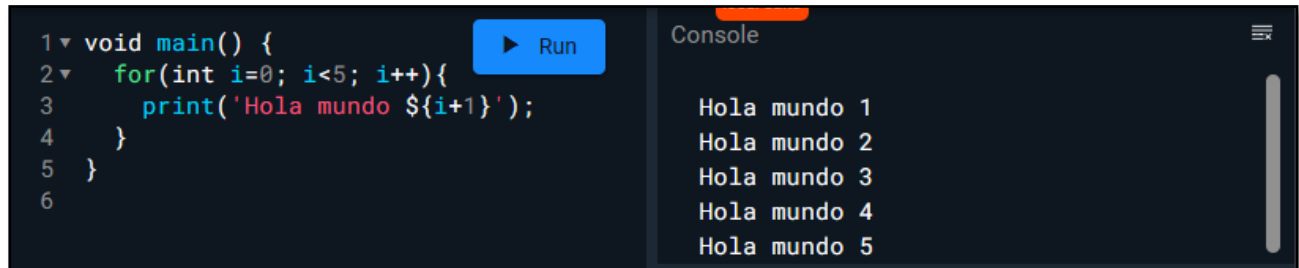


Figura I.1. Ejemplo "Hola Mundo" en DartPad

A la izquierda, se tiene el editor de código y un botón “Run”. Cuando es presionado, el resultado de la ejecución del código aparecerá a la derecha, mostrando tanto los resultados como los errores que se hayan encontrado.

Para el primer ejemplo, se eliminará el código predeterminado y se escribirá un programa Dart simple, el cual incluirá la declaración de variables y la concatenación de cadenas. A continuación, se presenta el código que deberá ser copiado en el editor.

```
void main() {  
  String nombre = "Dart";  
  print('¡Hola $nombre!');  
}
```

Al ejecutar código anterior, se tiene como resultado un mensaje de “¡Hola Dart!” a la derecha de la pantalla, como se puede apreciar en la siguiente imagen:

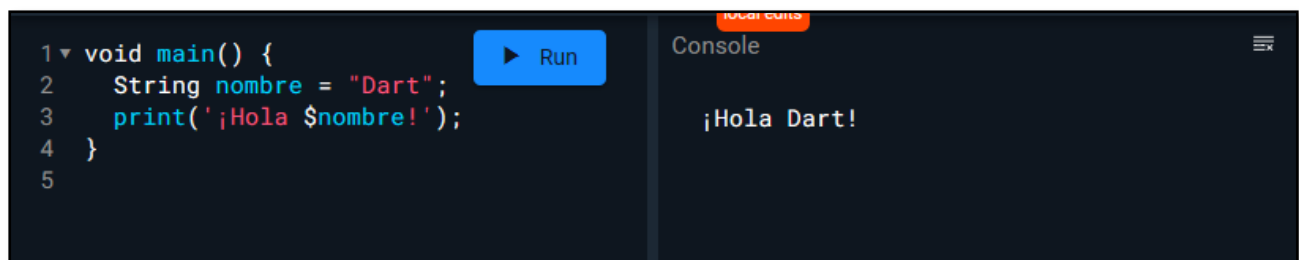


Figura I.2. Ejemplo "¡Hola Dart!"

En Dart, el inicio de cualquier programa se marca con la función `main()`, la cual es esencial y se encuentra en todas las aplicaciones desarrolladas con Flutter.

La variable llamada `nombre`, que es de tipo `String` y tiene como valor inicial "Dart", se declara en la línea 2 del código. Para delimitar cadenas se pueden emplear tanto comillas simples (') como dobles (").

La siguiente línea invoca al método de impresión, pasando una cadena como argumento. Lo notable es que, en vez de concatenar, se utiliza el signo `$` para insertar una variable directamente en la cadena, sin necesidad de cerrarla ni emplear el operador de concatenación `+`. Así, esto es equivalente a expresar lo siguiente:

```
print ("¡Hola " + nombre + "!");
```

Otra particularidad en Dart es la posibilidad de declarar variables de manera genérica, es decir, sin necesidad de especificar un tipo concreto. De esta forma, se puede obtener el mismo resultado mediante la escritura del código de la siguiente manera:

```
void main() {  
  var nombre = "Dart";  
  print("¡Hola $nombre!");  
}
```

Es importante proceder con precaución en este caso, dado que podría surgir una confusión al pensar que `nombre` es una variable dinámica, lo cual no es correcto. Para verificarlo, se sugiere asignar un valor de otro tipo a la variable y observar el resultado obtenido:

```
void main() {  
  var nombre = "Dart";  
  nombre = 42;  
  print("¡Hello $nombre!");  
}
```

Al intentar ejecutar este código, se recibirá un error de compilación, el cual se mostrará de la siguiente forma:

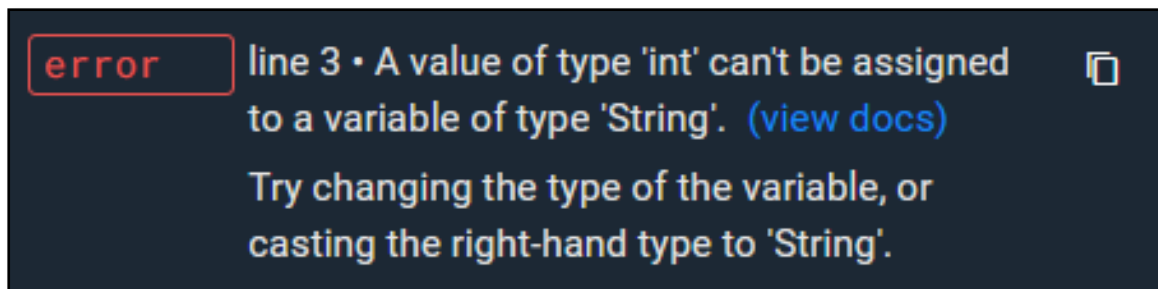


Figura I.3. Mensaje de Error (Un valor de tipo `int` no puede ser asignado en un tipo `String`)

El error recibido señala que no es posible asignar un entero a una variable de tipo cadena, lo que demuestra que declarar una variable con `var` no permite que la variable contenga distintos tipos de valores.

En realidad, si lo que se necesita es tener una variable que pueda contener distintos tipos de datos, se debe declarar de tipo dinámico, aunque se recomienda evitarlo en la mayoría de los casos o, como mínimo, manejarlo con mucha precaución. El código a continuación sirve como ejemplo de lo anteriormente descrito.

```
void main() {  
    dynamic nombre = "Dart";  
    nombre = 42;  
    print("¡Hola $nombre!");  
}
```

Tras llevar a cabo la ejecución del código, se podrá ver en la consola el texto "Hola 42". Esto indica que la variable `nombre`, que inicialmente se definió como una cadena de texto, ha sido modificada para almacenar un valor numérico entero. Esta transformación subraya la capacidad de manipular y cambiar el tipo de datos de una variable en el curso de la ejecución del programa.

Dado el enfoque actual en los números, se procederá a explorar más sobre este tema.

1.2.2 Distancia entre dos puntos

En este caso, se estudiará cómo Dart maneja los números, las funciones y los parámetros. Se examinará detalladamente cómo se pueden utilizar estos elementos dentro del lenguaje de programación para realizar operaciones matemáticas, definir bloques de código reutilizables a través de funciones y cómo estos pueden ser modificados o personalizados mediante el uso de parámetros, permitiendo así una mayor flexibilidad y adaptabilidad en la construcción de programas.

Para empezar, se debe aclarar que existen dos tipos de números en Dart:

- `int`: Que alberga valores enteros, limitados a 64 bits.
- `double`: Que guarda números en punto flotante de doble precisión, también en 64 bits.

Asimismo, Dart presenta el tipo `num`, que engloba a los tipos `int` y `double` bajo una misma categoría.

Para entender el manejo de números, se diseñará un programa en Dart para calcular la distancia entre dos puntos en el plano cartesiano. El ejemplo empleará conceptos de números, funciones y parámetros, fundamentándose en la fórmula para determinar la distancia entre dos puntos, la cual es:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Donde:

- d es la distancia entre dos puntos.
- (x_1, y_1) son las coordenadas del primer punto.
- (x_2, y_2) son las coordenadas del segundo punto.

Inicialmente se presenta el código de la función `calcularDistancia`:

```
double calcularDistancia(double x1, double y1, double x2, double y2) {  
    return sqrt(pow((x2 - x1), 2) + pow((y2 - y1), 2));  
}
```

La función *calcularDistancia* implementa la fórmula de la distancia entre dos puntos, se le pasan las coordenadas de los dos puntos como parámetros y devuelve el cálculo de la distancia.

Ahora se presenta el programa principal, el cual hace la invocación de la función que se acaba de explicar:

```
import 'dart:math';
void main() {
  // Coordenadas del primer punto
  double x1 = 4.0;
  double y1 = 2.0;
  // Coordenadas del segundo punto
  double x2 = 6.0;
  double y2 = 5.0;
  // Calcular la distancia entre los puntos
  double distancia = calcularDistancia(x1, y1, x2, y2);
  print("La distancia entre los puntos es: ${distancia.toStringAsFixed(2)}");
}
```

En el método *main*, se definen las coordenadas de dos puntos y se llama a *calcularDistancia* con esos valores. Finalmente, imprime el resultado formateado a dos decimales. Es importante notar que para usar *sqrt* (raíz cuadrada) y *pow* (potencia), se requiere importar el paquete *dart:math*.

De acuerdo con Google (2024a) la biblioteca *dart:math* proporciona funcionalidades comunes tales como el seno y coseno, el cálculo de valores máximo y mínimo, así como constantes matemáticas importantes como pi y e. Se destaca que en esta biblioteca están presentes las funcionalidades relacionadas con la radicación y la potenciación, operaciones que son necesarias en este caso para el ejercicio planteado.

La siguiente imagen muestra el código completo del ejemplo. Además, el resultado de la ejecución se encuentra en la parte derecha de la interfaz.



```
1 import 'dart:math';
2 void main() {
3   // Coordenadas del primer punto
4   double x1 = 4.0;
5   double y1 = 2.0;
6   // Coordenadas del segundo punto
7   double x2 = 6.0;
8   double y2 = 5.0;
9   // Calcular la distancia entre los puntos
10  double distancia = calcularDistancia(x1, y1, x2, y2);
11  print("La distancia entre los puntos es: ${distancia.toStringAsFixed(2)}");
12 }
13 // Función para calcular la distancia entre dos puntos
14 double calcularDistancia(double x1, double y1, double x2, double y2) {
15   return sqrt(pow((x2 - x1), 2) + pow((y2 - y1), 2));
16 }
```

La distancia entre los puntos es: 3.61

Figura I.4. Resultado del Ejemplo Calculadora de Área

Este ejemplo ilustra cómo se pueden utilizar números, funciones, parámetros y operaciones matemáticas básicas para realizar cálculos geométricos en Dart.

1.2.3 La reversa de una cadena

Dart admite los mismos bucles que muchos otros lenguajes influenciados por el lenguaje C, es decir, se tiene disponibles los bucles *for*, *while* y *do while*. En este ejemplo, se aplicará un bucle *for* para invertir una cadena.

Inicialmente se muestra el código que corresponde a la función principal:

```
void main() {  
    String myString = 'Hola a todos';  
    String result = reverse(myString);  
    print(result);  
}
```

No hay nada novedoso que destacar hasta aquí, simplemente se ha definido una cadena y se invocó a un método denominado *reverse*, que se debe programar para invertir la cadena que se envía como parámetro, para mostrarla en pantalla posteriormente.

Ahora, se expone el método *reverse*:

```
String reverse(String old) {  
    int length = old.length;  
    String res = '';  
    for (int i = length - 1; i >= 0; i--) {  
        res += old.substring(i, i + 1);  
    }  
    return res;  
}
```

Tomando en cuenta que este código tiene un mayor cuerpo, es oportuno, describir con detalle cada una de las líneas de código que lo componen:

Las siguientes líneas definen una función llamada *reverse* que toma un parámetro *old*, que es la cadena *String* que se desea invertir. Se verá también que la función devuelve un *String*.

```
String reverse(String old) {  
    ...  
}
```

La siguiente línea de código permite obtener la longitud de la cadena *old* y la almacena en la variable *length*.

```
int length = old.length;
```

Seguidamente, se crea una nueva cadena vacía llamada *res*, que se usará para almacenar el resultado.

```
String res = '';
```

El bucle *for* que se ha implementado, comienza desde el último carácter de la cadena *old* (en *length - 1*, ya que las cadenas están indexadas desde 0) y se ejecuta hacia atrás hasta el primer carácter (cuando *i* sea 0).

```
for (int i = length - 1; i >= 0; i--) {  
    ...  
}
```

Dentro del bucle, se toma cada carácter de *old* uno por uno, comenzando por el final y se lo añade a *res*. El método *substring(i, i + 1)* extrae el carácter en la posición *i* de *old*.

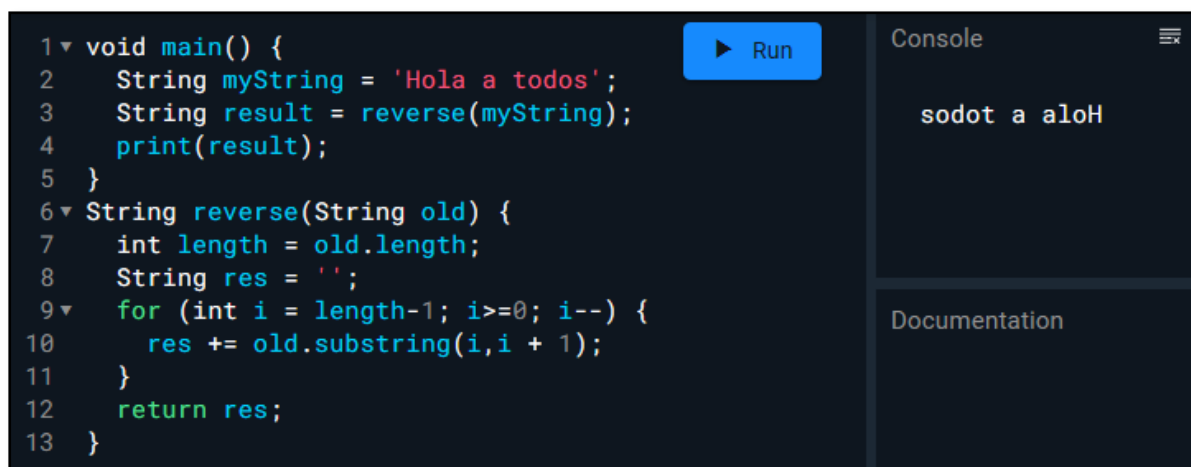
```
res += old.substring(i, i + 1);
```

Una vez que el bucle ha terminado, la cadena *res* contiene todos los caracteres de *old* en orden inverso. Esta cadena invertida se retorna como resultado de la función.

```
return res;
```

En resumen, la función *reverse* toma una cadena de texto como entrada, la recorre de atrás hacia adelante y construye una nueva cadena que es la inversión de la cadena original. Por ejemplo, dada la cadena “Hola a todos” como *old*, la función devolverá “sodot a aloH”.

El código completo y el resultado se muestra a continuación:



```
1 void main() {  
2   String myString = 'Hola a todos';  
3   String result = reverse(myString);  
4   print(result);  
5 }  
6 String reverse(String old) {  
7   int length = old.length;  
8   String res = '';  
9   for (int i = length-1; i>=0; i--) {  
10    res += old.substring(i,i + 1);  
11  }  
12  return res;  
13 }
```

Run

Console

sodot a aloH

Documentation

Figura I.5. Resultado del Ejemplo Reversa de una Cadena

No es común redactar un código como el anterior en una aplicación del mundo real. Debes tener en cuenta que los ejemplos de esta sección están diseñados para que te familiarices con el uso del lenguaje Dart. Sin embargo, es importante señalar que se puede lograr el mismo resultado simplemente escribiendo lo siguiente:

```
void main() {  
  String myString = 'Hola a todos';  
  String result = myString.split('').reversed.join();  
  print(result);  
}
```

A continuación, se presentan características ampliamente utilizadas a lo largo del libro: la sintaxis de flecha y el operador ternario.

1.2.4 Verificación de paridad

Se desea verificar si un número es par o impar y devolver la cadena correspondiente. En este ejemplo, se trabajará sobre ello.

```
void main() {  
  print(checkIfEvenOrOdd(5));  
}
```

Una función con la sintaxis tradicional sería la siguiente:

```
String checkIfEvenOrOdd(int number) {  
  if (number % 2 == 0) {  
    return 'Par';  
  } else {  
    return 'Impar';  
  }  
}
```

Sin embargo, en Dart tienes la alternativa de emplear la sintaxis de flecha (Arrow syntax), así como también se dispone del operador ternario. Por lo tanto, puedes escribir el código anterior de la siguiente manera:

```
String checkIfEvenOrOdd(int number) => number % 2 == 0 ? 'Par' : 'Impar';
```

Juntos, la sintaxis de flecha y el operador ternario forman una combinación poderosa y elegante, que se puede utilizar con regularidad en el desarrollo de aplicaciones móviles con Flutter.

1.2.5 Colección de superhéroes

Al adentrarse en el aprendizaje de un nuevo lenguaje de programación, uno de los conceptos fundamentales con los que comúnmente se encuentra son los arreglos (arrays). Estas estructuras de datos permiten almacenar múltiples valores en una única variable, facilitando así el manejo de colecciones de elementos. En Dart, los arreglos se manejan mediante la clase *List*, proporcionando un conjunto versátil de operaciones para manipular secuencias de datos, tales como agregar, eliminar y acceder a elementos. Esta característica es

esencial para la gestión eficiente de datos y la realización de operaciones complejas sobre colecciones de elementos.

En este ejemplo, se definirá una lista de héroes para devolverlos como una cadena, con el fin de trabajar con objetos de tipo lista.

```
void main() {  
  String myHeros = herosString();  
  print(myHeros);  
}  
String herosString() {  
  List<String> heros = [];  
  String herosString = '';  
  heros.add('Batman');  
  heros.add('Superman');  
  heros.add('Spiderman');  
  int i = 0;  
  while (i < heros.length) {  
    herosString += '${heros[i]} - ';  
    i++;  
  }  
  return herosString;  
}
```

El siguiente gráfico muestra el resultado final del código y el resultado en ejecución:

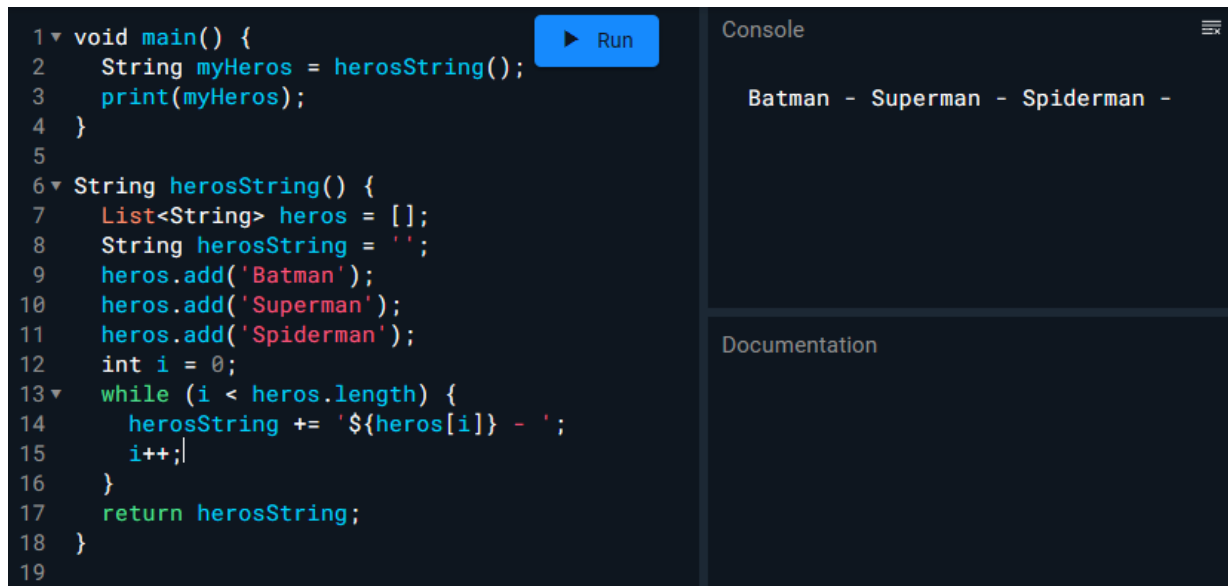


Figura I.6. Resultado del Ejemplo de Superhéroes

Sin embargo, esa no es la única forma de trabajar con listas. Existen alternativas que provienen de la programación funcional y que pueden hacer que el código Dart sea más simple y elegante. A continuación, se presentan esas funciones que pueden aprovecharse al usar listas en Dart.

El método `foreach()`

Los ciclos, tales como el *while*, son versátiles y pueden aplicarse al manejo de listas; sin embargo, las listas en sí ofrecen métodos particulares que mejoran la elegancia y legibilidad del código. Específicamente, el método *forEach* de una lista facilita la ejecución de una función sobre cada elemento de dicha lista, brindando una alternativa para prescindir del uso del bucle *while*. Así, es factible emplear el siguiente código como reemplazo para alcanzar un resultado idéntico:

```
void main() {
    String myHeros = herosString();
    print(myHeros);
}
String herosString() {
    List<String> heros = [];
    String herosString = '';
    heros.add('Batman');
    heros.add('Superman');
    heros.add('Spiderman');
    heros.forEach((hero) => herosString += hero + " - ");
    return herosString;
}
```

El método *foreach* toma una función como parámetro, esta puede ser anónima como lo es en el anterior ejemplo, donde la función toma un argumento *hero*, del mismo tipo de datos que la propia lista, entonces, como la lista es de cadenas *hero* también será una cadena.

La sintaxis flecha `=>` vista en el anterior ejemplo, establece el valor de la variable *herosString*.

El método `map()`

El método *map* se utiliza para aplicar una transformación a cada elemento de una lista, generando como resultado una nueva lista que contiene los elementos transformados. Este proceso permite, por ejemplo, modificar la lista original sin alterarla directamente, manteniendo así la inmutabilidad de los datos. En el ejemplo proporcionado, se convierten todos los nombres en la lista de superhéroes a letras mayúsculas, demostrando cómo *map* facilita la realización de operaciones de transformación de manera concisa y eficiente.

```
void main() {
    String myHeros = herosString();
    print(myHeros);
}
String herosString() {
    List<String> heros = [];
    heros.add('Batman');
    heros.add('Superman');
    heros.add('Spiderman');
    var capitalHeros = heros.map((hero) => hero.toUpperCase());
    return capitalHeros.toString();
}
```

La parte central del código es la siguiente línea:

```
var capitalHeros = heros.map((hero) => hero.toUpperCase());
```

Aquí se observa el método *map* transformando cada elemento de la lista a mayúsculas con la instrucción *hero.toUpperCase()*. El resultado final se pasa a una nueva variable, llamada *capitalHeros*.

```
return capitalHeros.toString();
```

El método *toString()* transforma una lista en una cadena, por lo que, el resultado que se verá impreso en pantalla será el siguiente: (BATMAN, SUPERMAN, SPIDERMAN)

El método where()

Se presenta otro método en esta breve descripción, el cual es el método *where*. Para comprenderlo, se sugiere modificar el código de la siguiente manera:

```
void main() {
    String myHeros = herosString();
    print(myHeros);
}
String herosString() {
    List<String> heros = [];
    heros.add('Batman');
    heros.add('Superman');
    heros.add('Spiderman');
    var pHeros = heros.where((hero) => hero.contains('p'));
    return pHeros.toString();
}
```

El método *where* filtra y devuelve únicamente aquellos elementos de una lista que cumplen con una condición especificada en la función de prueba, como en este caso con *hero.contains('p')*, que selecciona solo los héroes cuyos nombres incluyen la letra "p". Por tanto, el resultado que se mostrará en pantalla será: (Superman, Spiderman).

Dentro del amplio abanico de métodos disponibles para manipular listas, se encuentran varios que facilitan la tarea de ordenar, transformar y localizar elementos específicos. Aunque a lo largo de este libro se explorarán otros métodos, por el momento, el uso de *foreach*, *map*, y *where* ya ofrece una sólida base para empezar a trabajar con listas de manera efectiva.

1.2.6 Clase para automóviles

Según la documentación de Dart (Google, 2024a):

Dart es un lenguaje orientado a objetos con clases y herencia basada en mixins. Cada objeto es una instancia de una clase y todas las clases excepto Null descienden de Object. La herencia basada en mixins significa que, aunque cada clase (excepto la clase superior, Object?) tiene exactamente una superclase, un cuerpo de clase puede reutilizarse en múltiples jerarquías de clases. Los métodos de extensión son una forma de añadir funcionalidad a una clase sin cambiar la clase o crear una subclase. Los modificadores de clase te permiten controlar cómo las bibliotecas pueden subtipar una clase. [Traducción propia]

En el contexto de Dart, un lenguaje de programación orientado a objetos, las clases y los objetos son componentes esenciales para la construcción de programas. Las clases actúan como plantillas o moldes que definen la estructura y el comportamiento de los objetos, que son instancias concretas de esas clases. Esta metodología de programación permite organizar y modularizar el código de manera eficiente, facilitando la reutilización y la mantenibilidad del software. Por lo tanto, comprender y dominar el uso de clases y objetos en Dart es fundamental para desarrollar aplicaciones eficaces y escalables, especialmente en el contexto de desarrollo de aplicaciones móviles con Flutter.

En el siguiente ejemplo, se ilustra el uso de clases y objetos en Dart. Se creará una clase Car para representar un automóvil con algunas propiedades básicas.

```
class Car {
  String make;
  String model;
  int year;
  Car(this.make, this.model, this.year);
  void displayCarInfo() {
    print('Automovil: $make $model, Año: $year');
  }
}
```

Se completa el código con el método *main*, en el cual se crea una instancia de la clase mencionada.

```
void main() {  
    Car myCar = Car('Toyota', 'Corolla', 2020);  
    myCar.displayCarInfo();  
}
```

En el ejemplo, se puede observar un constructor, la tarea de asignar un argumento de constructor a una variable de objeto es algo que probablemente se hace muy a menudo en programación orientada a objetos. Dart lo hace extremadamente fácil como se puede observar en el ejemplo. Es decir, en lugar de la sintaxis tradicional:

```
Car(String make, String model, int year) {  
    this.make = make;  
    this.model = model;  
    this.year = year;  
}
```

En Dart se escribe de la siguiente manera:

```
Car(this.make, this.model, this.year);
```

Getters y Setters

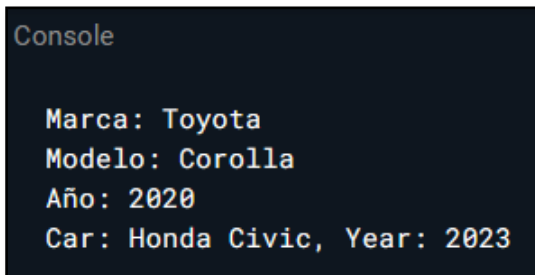
Los métodos conocidos como getters y setters se emplean para asegurar la integridad de los datos dentro de las clases. Un getter retorna el valor actual de un atributo de una instancia de clase, mientras que un setter permite modificar o asignar un nuevo valor a dicho atributo. Esto facilita la implementación de validaciones o comprobaciones antes de acceder (con getters) o modificar (con setters) los atributos de las clases. Para definir estos métodos, se utilizan las palabras reservadas *get* y *set*, seguidos del nombre del atributo. Mientras que el getter especifica y retorna un valor del tipo definido, el setter, por su parte, no retorna ningún valor (void).

Ahora, el código anterior se modifica para emplear getters y setters y queda de la siguiente manera:


```
class Car {
    String _make;
    String _model;
    int _year;
    Car(this._make, this._model, this._year);
    String get make => _make; // Getter para 'make'
    set make(String value) { // Setter para 'make'
        _make = value;
    }
    String get model => _model; // Getter para 'model'
    set model(String value) { // Setter para 'model'
        _model = value;
    }
    int get year => _year; // Getter para 'year'
    set year(int value) { // Setter para 'year'
        _year = value;
    }
    void displayCarInfo() {
        print('Car: $_make $_model, Year: $_year');
    }
}

void main() {
    Car myCar = Car('Toyota', 'Corolla', 2020);
    // Usar getters
    print('Marca: ${myCar.make}');
    print('Modelo: ${myCar.model}');
    print('Año: ${myCar.year}');
    // Usar setters
    myCar.make = 'Honda';
    myCar.model = 'Civic';
    myCar.year = 2023;
    // Mostrar información actualizada
    myCar.displayCarInfo();
}
```

El resultado de la ejecución del anterior código es el siguiente:



```
Console

Marca: Toyota
Modelo: Corolla
Año: 2020
Car: Honda Civic, Year: 2023
```

Figura I.7. Resultado del Ejemplo Clase Automóvil

Con las clases y los objetos, se dispone de todas las herramientas del lenguaje Dart para iniciar un proyecto de Flutter. A lo largo de este libro, se explorarán numerosas características y temas adicionales en Dart, los cuales serán abordados conforme sean necesarios para los ejercicios de la obra.

1.3 Configuración del entorno de desarrollo en Windows

Configurar el entorno de desarrollo para Flutter es un proceso esencial para comenzar a trabajar en proyectos de aplicaciones móviles. A continuación, se presenta una guía detallada paso a paso para configurar el entorno:

1.3.1 Requisitos del sistema

Para instalar y ejecutar Flutter tomando en cuenta un ambiente de desarrollo bajo sistema operativo Windows, según indica Google (2024b) una computadora deberá contar con el siguiente **software**:

- **Sistema operativo:** Windows 10 o posterior (64 bits).
- **Herramientas:** Flutter requiere que las herramientas siguientes estén instaladas en el ordenador.
 - Se necesita Windows PowerShell 5.0 o una versión más reciente, el cual viene preinstalado en Windows 10.
 - Es necesario contar con Git para Windows versión 2.x, configurado para permitir el uso de Git desde el símbolo del sistema de Windows. En caso de que Git para Windows ya esté instalado, es decisivo verificar que se puedan llevar a cabo comandos de Git a través del símbolo del sistema o PowerShell.

En cuanto al **hardware**, según la documentación oficial de Flutter (Google, 2024b), la computadora deberá cumplir con los siguientes requisitos:

- **Memoria RAM:** Mínimo 8 GB de RAM (Random Access Memory) (16 GB recomendado)
- **Disco duro:** Un disco duro SSD (Solid State Drive) (recomendado)
- **Espacio en disco:** Mínimo 4 GB, se recomienda 52 GB de espacio libre (sin considerar el espacio en disco para IDE/herramientas).
- **Procesador:** Mínimo de 4 núcleos, se recomienda de 8 núcleos.

Si bien no se constituyen en requisitos obligatorios, disponer de menos podría resultar en un incremento del tiempo de espera en comparación con el tiempo de trabajo efectivo.

1.3.2 Obtención de Flutter SDK

El SDK de Flutter es un kit de desarrollo de software creado por Google, es la herramienta que se requiere para la creación de las aplicaciones móviles. Para instalarlo en la computadora, se debe seguir los siguientes pasos:

1. Obtener Flutter SDK:

- a. Visitar la página oficial de Flutter en la sección de instalación⁴.

⁴ <https://docs.flutter.dev/get-started/install>

- b. Descargar la versión más actual y estable del SDK de Flutter específicamente diseñada para sistemas operativos Windows, asegurando obtener las últimas mejoras y características disponibles.
- c. El archivo descargado se debe extraer en la computadora (recomendable hacerlo en: D:\src\flutter u otra unidad donde no este instalado el sistema operativo).

2. Actualizar el Path del Sistema:

- a. En la barra de búsqueda que se encuentra en la parte inferior del escritorio, se debe escribir “variables” para seleccionar la opción “Editar las variables de entorno del sistema”, esto abrirá la ventana “Propiedades del sistema”. En esa ventana se debe presionar en el botón “Variables de entorno”.

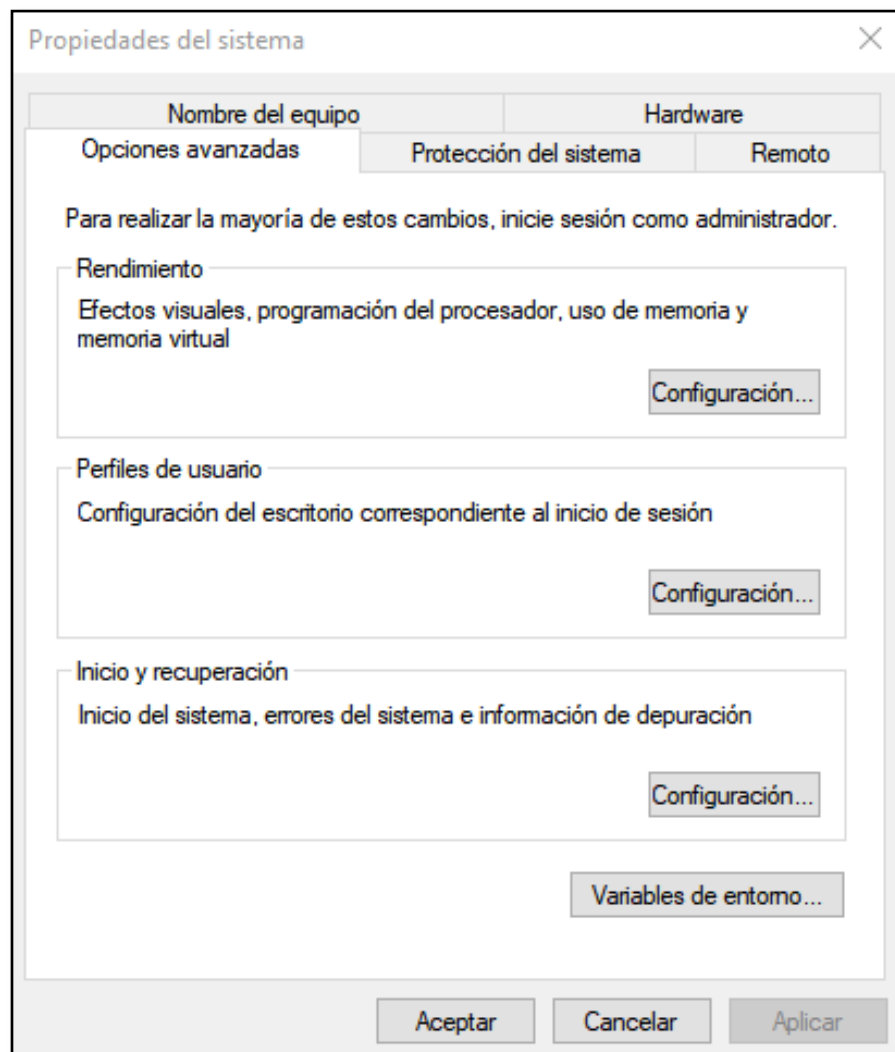


Figura I.8. Propiedades del Sistema

- b. En el siguiente cuadro de diálogo, se deber seleccionar la variable “Path” de la sección “Variables de usuario para ...” y presionar en el botón Editar.

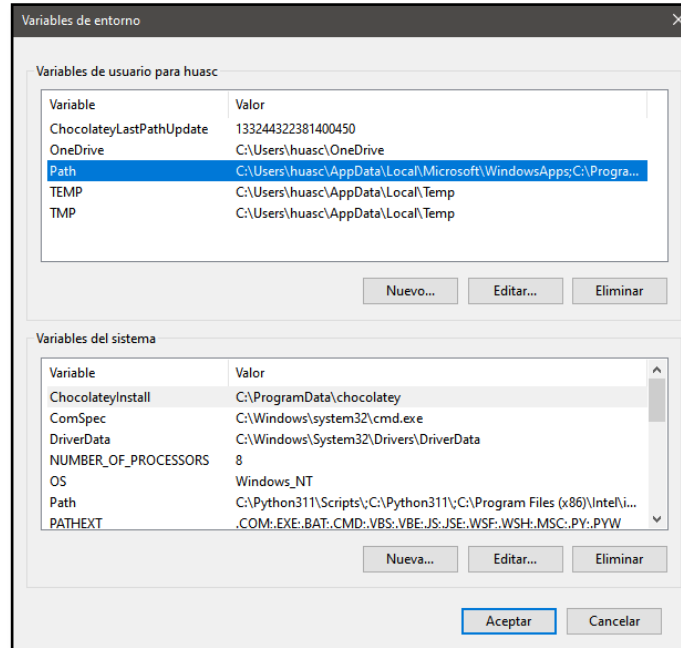


Figura I.9. Variables de Entorno

- c. Una vez seleccionado “Path” en la lista, se debe presionar en el botón “Editar” y luego en “Nuevo” para agregar “D:\flutter\bin” (se debe tener cuidado de poner la unidad donde se haya descomprimido la SDK), luego selecciona “Aceptar”. Flutter ahora debería agregarse a la lista de variables de entorno como se puede apreciar a continuación.

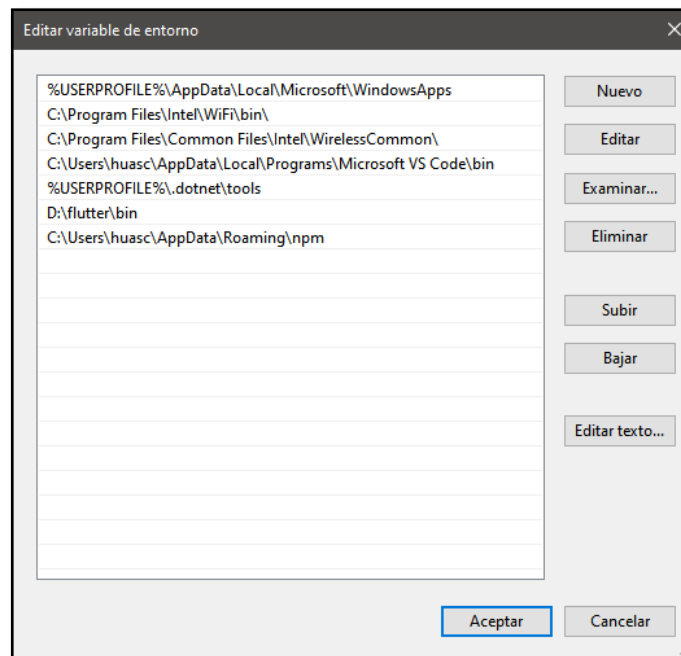


Figura I.10. Editar Variables de Entorno

- d. Ahora simplemente se debe reiniciar la computadora.

1.3.3 Verificación del entorno con Flutter Doctor

La herramienta "Flutter Doctor" se convierte en una aliada fundamental para cualquier desarrollador. Su función es proporcionar una lista detallada de los pasos necesarios para garantizar el correcto funcionamiento de Flutter en el sistema. Durante el proceso de instalación, es recomendable utilizar Flutter Doctor como guía para verificar la configuración y las actualizaciones necesarias.

Para utilizarlo, se debe abrir la terminal preferida (Símbolo de Sistema, Windows PowerShell u otro) y escribir el siguiente comando:

```
flutter doctor
```

Este comando verifica el estado de la computadora en relación con los requisitos necesarios para el funcionamiento de Flutter y proporciona un informe detallado del estado de la instalación. Se recomienda revisar cuidadosamente el resultado para identificar cualquier software que necesite ser instalado o cualquier tarea adicional que deba ser realizada.

Se continúa con la instalación; ahora es momento de proceder con la instalación de Android Studio.

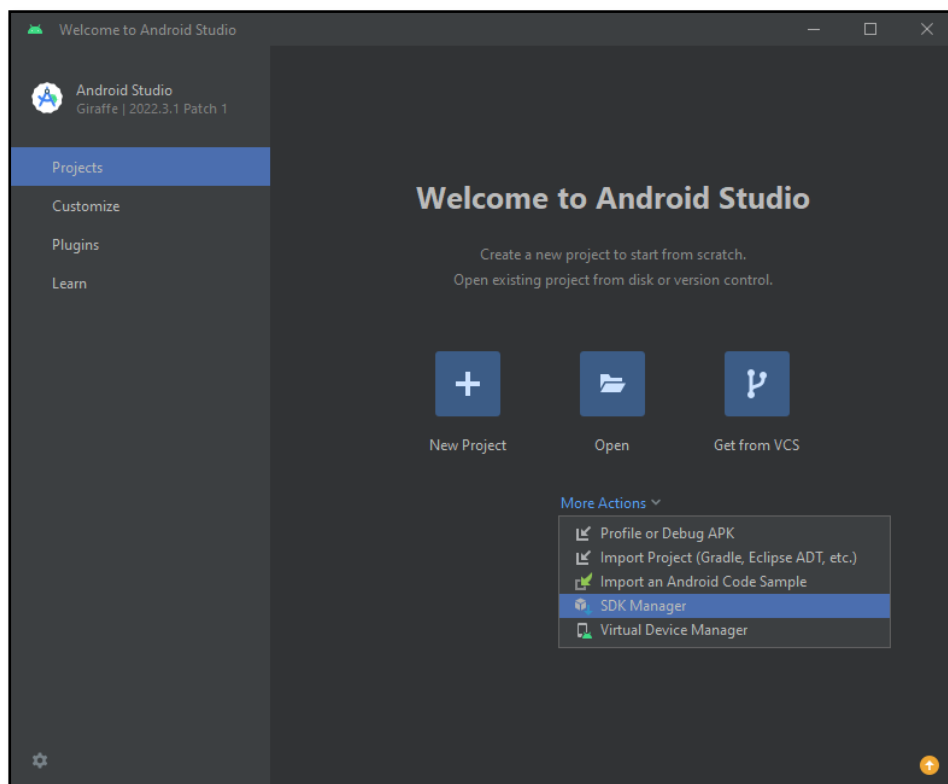
1.3.4 Instalación de Android Studio

Android Studio es el entorno de desarrollo integrado oficial para Android, proporcionado por Google y es necesario para el desarrollo con Flutter principalmente porque ofrece el SDK de Android, que se trata de herramientas específicas para el desarrollo de aplicaciones Android, emuladores para probar aplicaciones en diferentes dispositivos virtuales y una integración fluida con Flutter y Dart a través de plugins específicos. Facilita la configuración inicial del entorno de desarrollo y la gestión de dependencias.

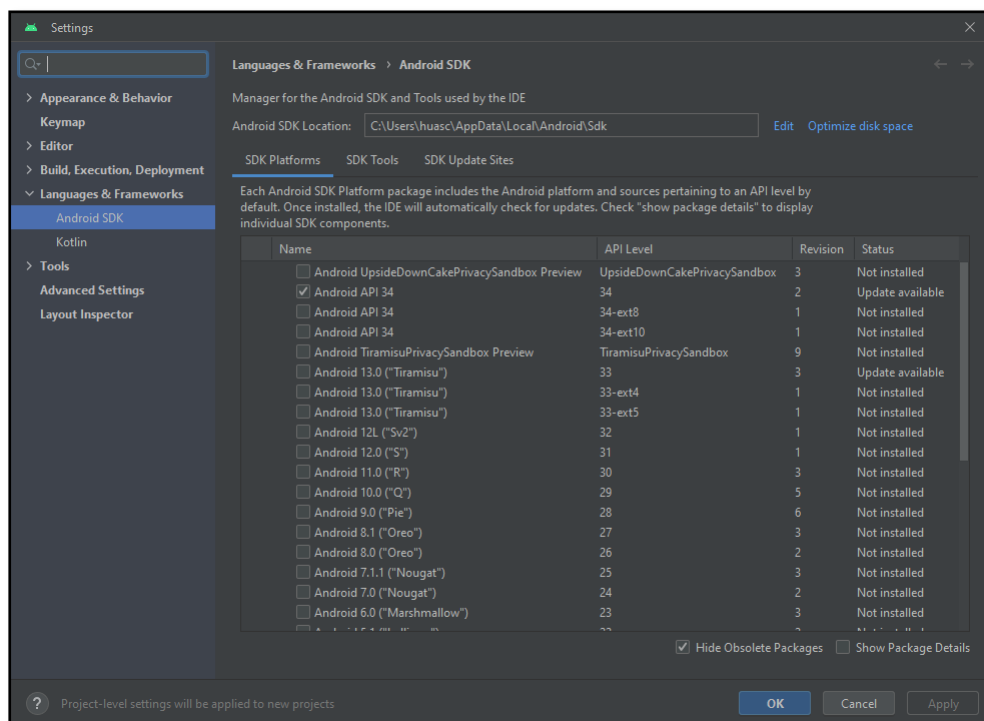
Para su instalación, se sigue los siguientes pasos:

1. Descargar Android Studio desde su sitio web⁵.
2. Ejecutar el instalador de Android Studio y seguir los pasos que indica el "Asistente de configuración de Android Studio" para completar la instalación.
3. Una vez instalado Android Studio, se debe descargar al menos un SDK de Android (recomendado el más reciente). Para ello, se debe ejecutar las siguientes tareas: Abrir el SDK Manager, para ello se debe abrir "Android Studio" > "More Actions" > "SDK Manager".

⁵ <https://developer.android.com/studio?hl=es-419>

*Figura I.11. Apertura del SDK Manager*

4. En la pestaña “SDK Platforms” se debe verificar que se tenga instalado el último “Android SDK”, a la fecha de escritura de este libro es “Android API 34”. Instalarlo en caso de no tenerlo.

*Figura I.12. Settings - SDK Platforms*

- En la pestaña “SDK Tools” se debe verificar que este instalado o en su caso instalar “Android SDK Build-Tools”, “Android SDK Command-line Tools” y “Android SDK Platform-Tools”.

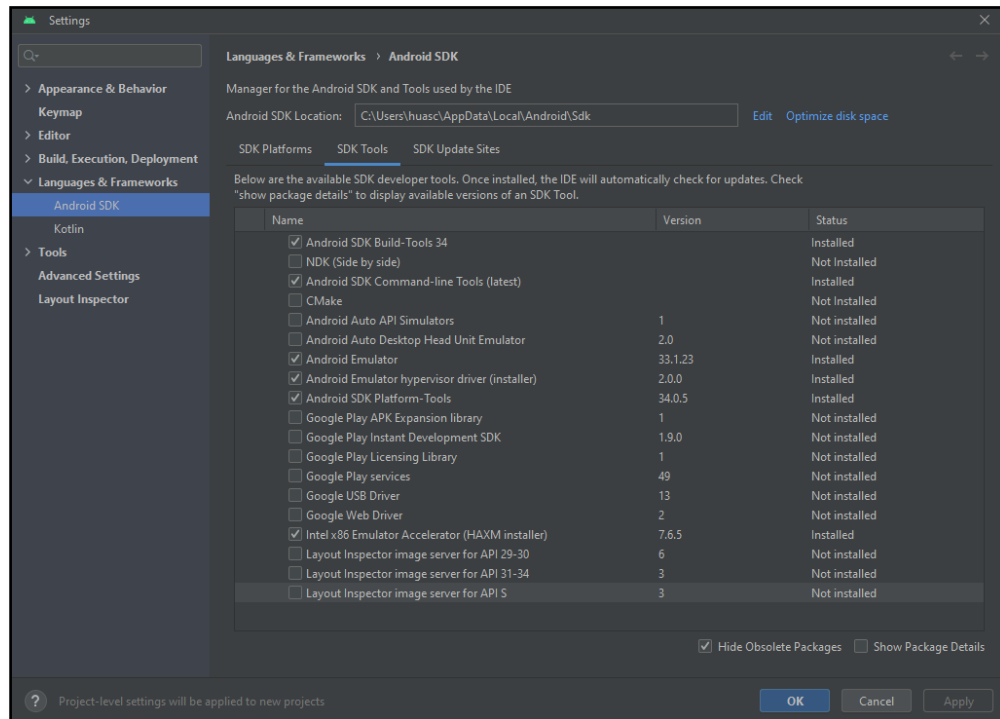


Figura I.13. Settings - SDK Tools

- Se debe ejecutar nuevamente “flutter doctor” para verificar que Flutter haya detectado la instalación de Android Studio. En caso de que Flutter no logre localizarla, se puede utilizar el siguiente comando para configurar el directorio de instalación de Android Studio.

```
flutter config --android-studio-dir=<directory>
```

Cabe señalar que se requiere un dispositivo en el cual observar las aplicaciones que se construirán, por eso a continuación, se explican los pasos para la creación de un emulador Android.

1.3.5 Creación de un emulador de Android

Para ejecutar las aplicaciones que se vayan creando, se necesitará algún tipo de dispositivo donde visualizarlas. En el contexto de desarrollo para Android, la experiencia con un dispositivo físico es insuperable. Por tanto, si se dispone de acceso a un dispositivo Android real, se aconseja emplearlo para las tareas de desarrollo en la mayor medida posible.

Sin embargo, alternativamente es posible utilizar un emulador de Android, ya que a menudo es más sencillo tener un dispositivo virtual al lado del código que permita ver los resultados según las modificaciones que se vayan haciendo.

Se deben seguir estos pasos para configurar el primer emulador:

1. Abrir “Virtual Device Manger” de Android Studio, la siguiente imagen muestra cómo hacerlo.

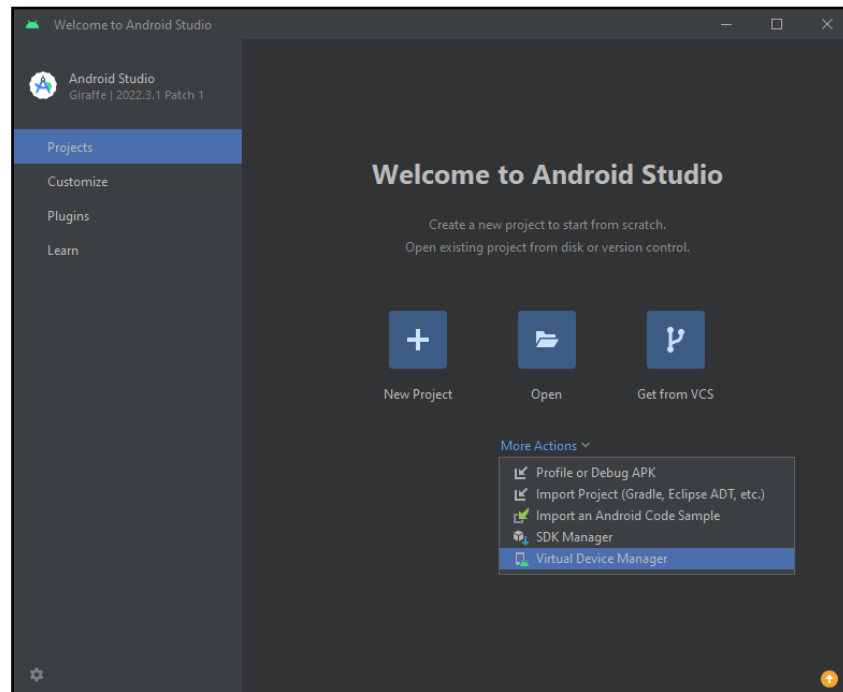


Figura I.14. Apertura de Virtual Device Manager

2. Luego seleccionar el botón “Create Device” para comenzar a construir el dispositivo virtual.
3. La siguiente pantalla permite configurar el hardware de Android que se desea emular. Se recomienda emplear un dispositivo Pixel.

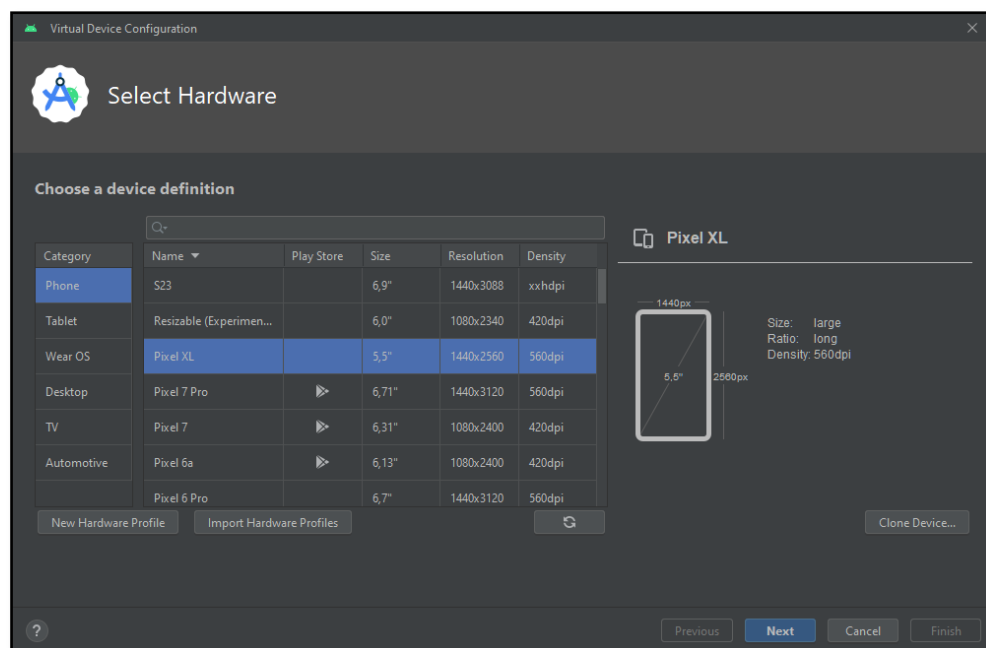


Figura I.15. Virtual Device Configuration

- Ahora seleccionar una imagen de sistema, en la mayoría de los casos, la imagen más reciente será suficiente (a la fecha API 34). Cada una de estas imágenes tiene un tamaño de varios gigabytes (GB), así que se debe descargar sólo lo que se necesite.

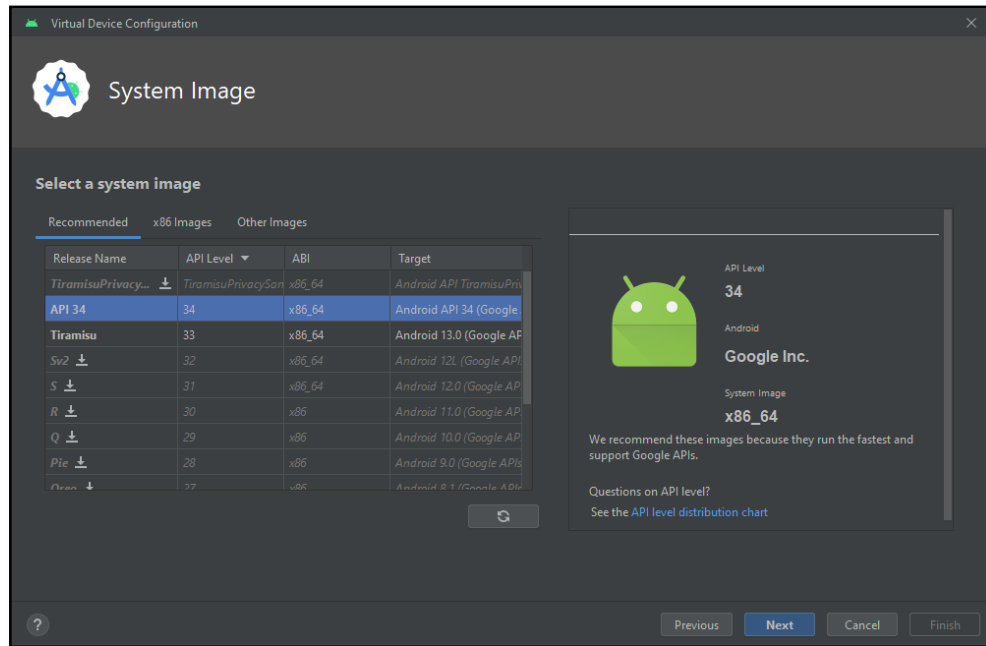


Figura I.16. System Image

- Haz clic en “Siguiente” para crear tu emulador, a estas alturas, es posible iniciar el emulador si lo desea, pero no es necesario por ahora.
- Una vez más, se debe ejecutar “flutter doctor” para comprobar el entorno.

1.3.6 Licencias de Android

Antes de utilizar Flutter, se debe aceptar las licencias de la plataforma SDK de Android, este paso se debe hacer después de haber instalado las herramientas citadas anteriormente.

- Iniciar la terminal y correr el siguiente comando para aceptar las licencias.

```
flutter doctor --android-licenses
```

- Se debe leer atentamente los términos de cada licencia antes de aceptarlos.
- Una vez se haya terminado de aceptar las licencias, ejecutar “flutter doctor” nuevamente para confirmar si el sistema está listo para usar Flutter o en su caso solucionar los inconvenientes que se puedan presentar.

1.3.7 Requisitos adicionales para escritorio de Windows

Cabe señalar, que para desarrollar aplicaciones móviles Android no es necesario la instalación de los requisitos para escritorio de Windows. Por otro lado, si lo que se desea es desarrollar aplicaciones para escritorio de Windows, se requiere lo siguiente además del SDK de Flutter.

- “Visual Studio 2022” o “Visual Studio Build Tools 2022”
- Instalar también “Desktop development with C++” incluidos todos sus componentes predeterminados, eso se puede instalar desde el asistente de instalación de Visual Studio.

1.3.8 Instalación de Visual Studio Code

Visual Studio Code es un editor de código fuente ligero pero poderoso que juega un rol fundamental en el desarrollo de aplicaciones con Flutter por las siguientes razones:

- Proporciona un entorno de edición de código altamente eficiente para Dart. Además, ofrece características como resaltado de sintaxis, formateo automático de código y sugerencias inteligentes, que facilitan la escritura y comprensión del código.
- Visual Studio Code se integra estrechamente con Flutter y Dart, esto te permite usar funcionalidades específicas de Flutter, como la recarga en caliente (hot reload), que actualiza inmediatamente la interfaz de usuario cuando se hacen cambios en el código.
- Tiene herramientas de depuración para aplicaciones Flutter, con esto se puede establecer puntos de interrupción, inspeccionar variables, ver pilas de llamadas y realizar un seguimiento del flujo de ejecución del programa. Esto es decisivo para encontrar y corregir errores en la aplicación.
- Facilita la gestión de dependencias de Flutter, ya que permite agregar, actualizar y eliminar paquetes de Flutter directamente desde Visual Studio Code.
- Se integra con sistemas de control de versiones como Git, permitiendo gestionar versiones y colaborar con otros en proyectos de Flutter.
- Se puede personalizar el entorno de desarrollo añadiendo extensiones que mejoran la productividad, como linters, formateadores de código y extensiones específicas de Flutter y Dart.
- Permite ejecutar comandos Flutter directamente desde el editor, facilitando tareas como la ejecución de pruebas, actualización de paquetes y más.

En resumen, Visual Studio Code actúa como un centro de control para el desarrollo de Flutter, proporcionando las herramientas necesarias para escribir, depurar, probar y mantener aplicaciones Flutter de manera eficiente y efectiva. Su naturaleza extensible y su integración fluida con Flutter y Dart lo convierten en una excelente opción.

Para contar con esta herramienta, se pueden seguir los siguientes pasos:

1. Se debe obtener Visual Studio Code descargándolo desde su sitio web oficial⁶.
2. Instalar siguiendo los pasos que indica el asistente de instalación.
3. Instalar la extensión de Flutter⁷ y Dart⁸ desde el marketplace de extensiones.

⁶ <https://code.visualstudio.com/>

⁷ <https://marketplace.visualstudio.com/items?itemName=Dart-Code.flutter>

⁸ <https://marketplace.visualstudio.com/items?itemName=Dart-Code.dart-code>

1.3.9 Creación y ejecución de un proyecto en Visual Studio Code

A estas alturas, ya se tendría el entorno de desarrollo de Flutter instalado y configurado en la computadora, lo que permite trabajar en la primera aplicación.

El propósito de esta actividad, más allá de comprender el código que se escribirá, es aprender cómo crear y ejecutar un proyecto. Para lograr ese propósito, se realizarán los siguientes pasos:

1. Abrir Visual Studio Code
 - a. Asegurarse de que las extensiones de Flutter y Dart estén instaladas, caso contrario, instalar esas extensiones.
2. Crear un nuevo proyecto Flutter
 - a. En el menú principal de Visual Studio, elegir la opción “Ver” > “Paleta de Comandos” o presionar las teclas Ctrl + Shift + P para acceder a “Paleta de Comandos” desde el teclado.
 - b. Escribir “flutter” y seleccionar “Flutter: New Project”, luego se debe presionar “Enter”.

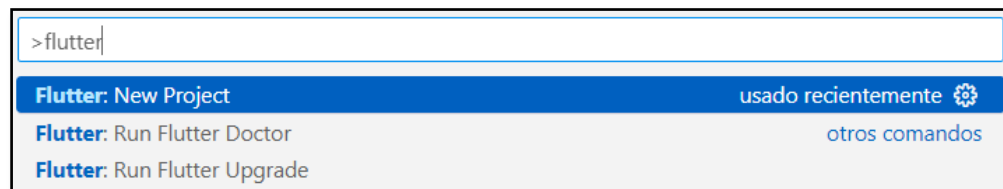


Figura I.17. Flutter: New Project

- c. En el diálogo que aparece a continuación, se debe elegir “Application” y luego presionar “Enter”.

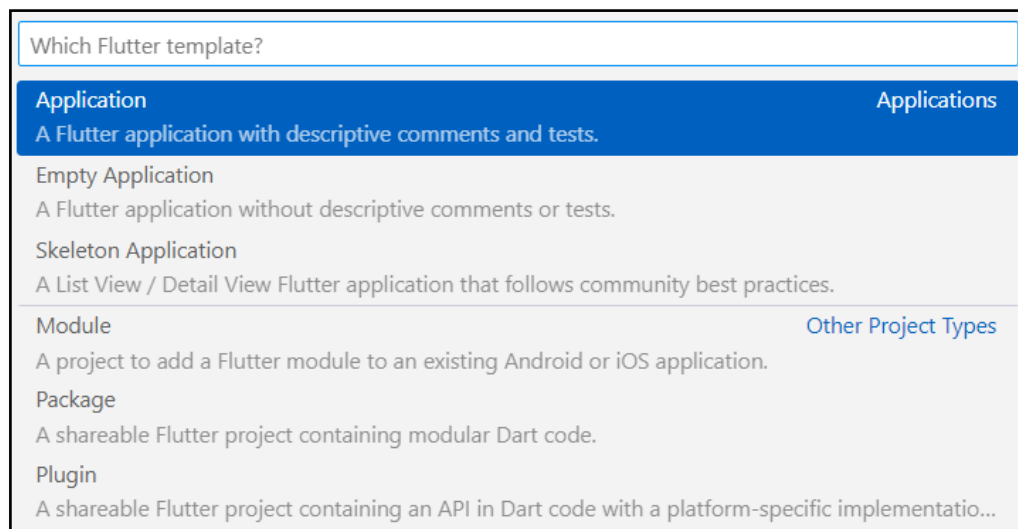
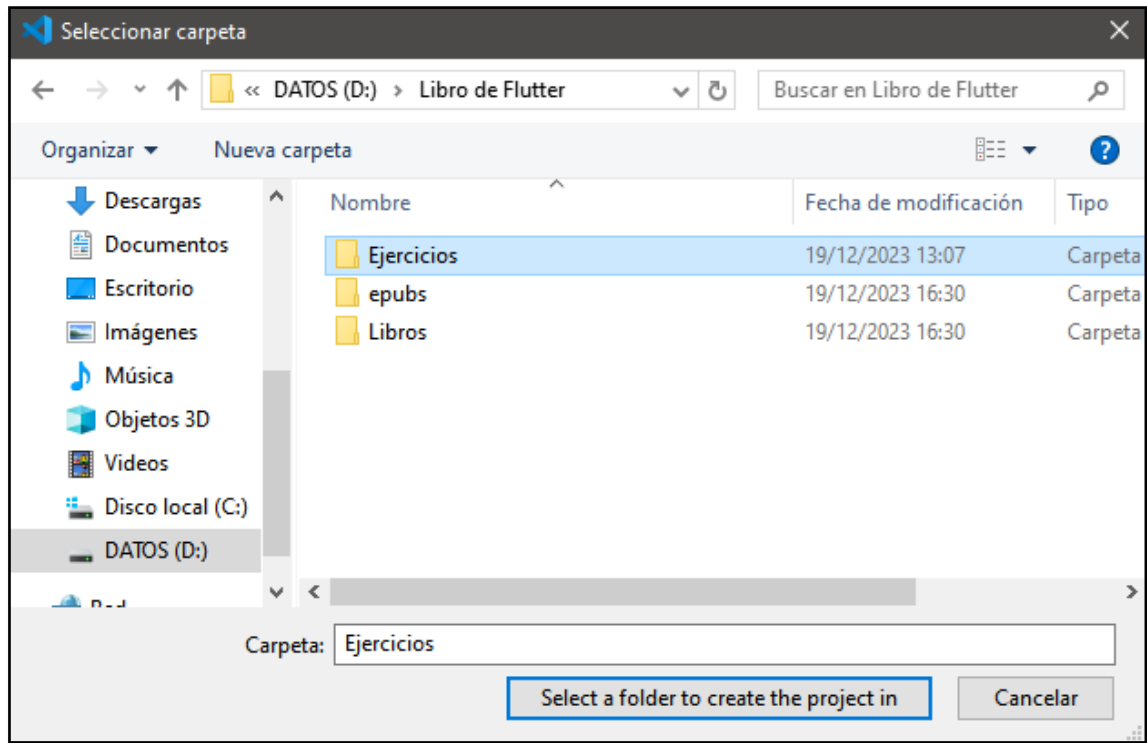
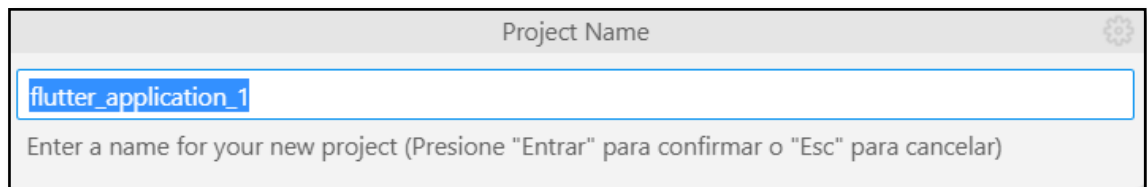


Figura I.18. Which Flutter template?

- d. Elegir una carpeta como ubicación para el proyecto y presionar en el botón “Select a folder to create the Project in”.

*Figura I.19. Seleccionar carpeta*

- e. Ingresar un nombre para el proyecto.

*Figura I.20. Project Name*

3. Explorar la estructura de carpetas y archivos del proyecto.
- a. En la barra lateral, se encuentra la estructura por defecto que tiene un proyecto de Flutter. Por ahora, simplemente se debe abrir el archivo “main.dart” que está en la carpeta “lib”.

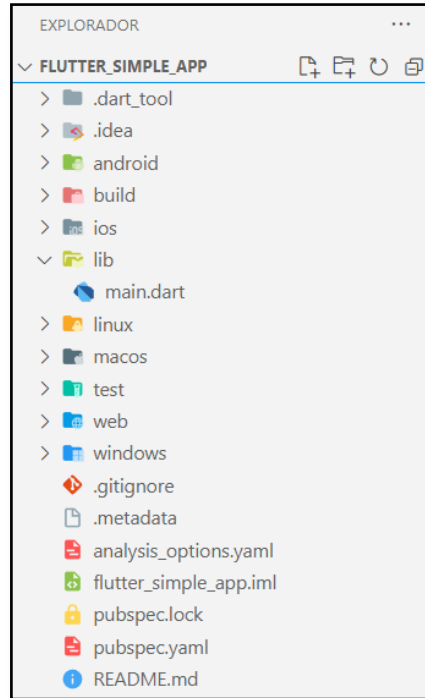


Figura I.21. Explorador

4. Ahora se ejecutará el proyecto:
 - a. Iniciar el emulador directamente desde Visual Studio Code presionando Ctrl + Shift + P.
 - b. Luego escribir “flutter”, seleccionar “Flutter: Select Device” y presionar “Enter”.

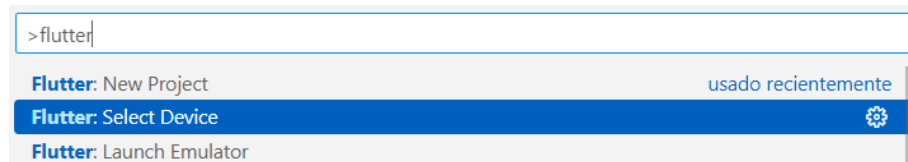


Figura I.22. Flutter: Select Device

- c. Elegir el emulador o dispositivo físico de preferencia y presiona “Enter”.

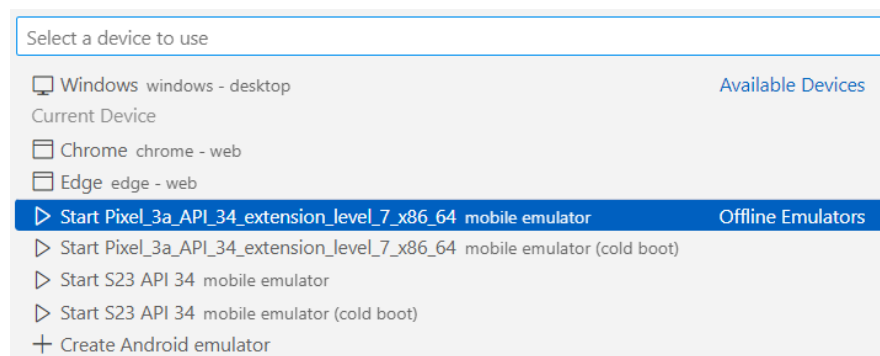


Figura I.23. Select a device to use

Entre las opciones se puede abrir un emulador o en su caso se puede seleccionar un dispositivo físico (eso en caso de tener conectado un dispositivo físico que además este en modo desarrollador).

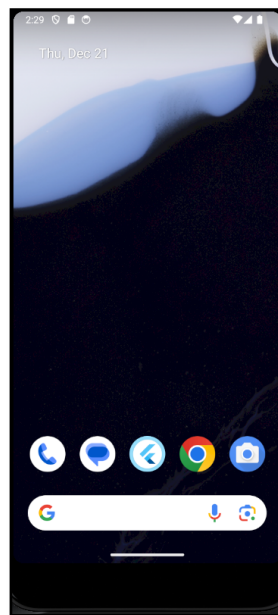


Figura I.24. Emulador en Funcionamiento

- d. Verificar que este abierto el archivo “main.dart”.
- e. Elegir la opción “Ejecutar” > “Iniciar depuración” en la barra superior o presiona F5 para iniciar la depuración.

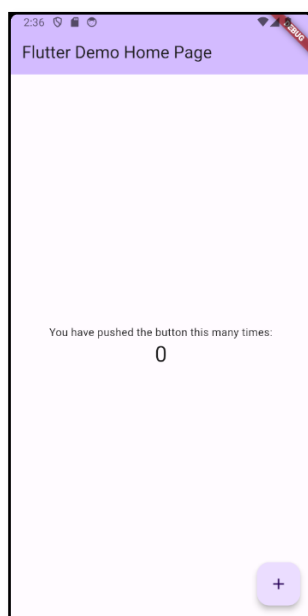


Figura I.25. Proyecto de Prueba en Ejecución

La aplicación se compilará y se ejecutará en el emulador o dispositivo seleccionado. Visual Studio Code mostrará la consola de salida donde se podrá ver los logs y mensajes de la aplicación.

Listo, ya se tiene el entorno de trabajo en funcionamiento y preparado para empezar a probar los programas que se desarrollaran en el libro.

1.3.10 Consejos adicionales

Al margen de lo realizado, existen algunos consejos a considerar que pueden ayudar a tener un trabajo mucho más liviano.

- **Actualizaciones:** Se puede mantener el entorno de Flutter actualizado con el comando “flutter upgrade”.
- **Hot Reload:** Visual Studio Code soporta Hot Reload (actualización rápida cada vez que guardas los cambios).
- **Hot Restart:** Se puedes usar Hot Restart para reconstruir completamente el estado de la aplicación.
- **Depuración:** Se debe utilizar la funcionalidad integrada de depuración de Visual Studio Code para establecer puntos de interrupción y examinar el estado de la aplicación.
- **Extensiones Útiles:** Considerar instalar extensiones adicionales como Flutter widget Snippets⁹ para mejorar la productividad.
- **Uso de Dependencias:** Considerar la instalación de la extensión Pubspec Assist¹⁰ para gestionar fácilmente las dependencias en los proyectos.

1.4 Ejercicio: crea y ejecuta una aplicación Flutter simple

Este ejercicio tiene como objetivo familiarizarse con la codificación de una aplicación Flutter. Se creará una aplicación simple (el clásico “Hola Mundo”) que muestra un mensaje en pantalla, el resultado se puede ver en el siguiente gráfico.

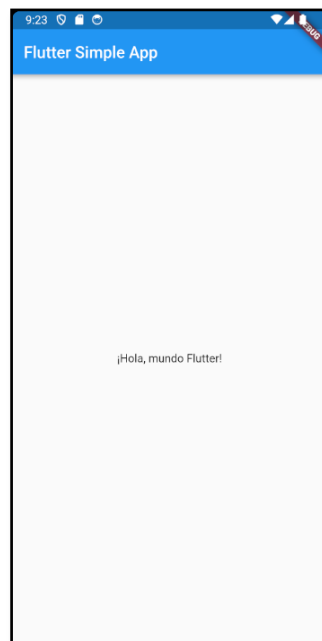


Figura I.26. Resultado de Flutter Simple App

⁹ <https://marketplace.visualstudio.com/items?itemName=alexisvt.flutter-snippets>

¹⁰ <https://marketplace.visualstudio.com/items?itemName=jeroen-meijer.pubspec-assist>

Requisitos Previos

- ❖ Asegurarse de tener el entorno de trabajo instalado y configurado correctamente.

Pasos del Ejercicio

1. Crear un nuevo proyecto Flutter esta vez con el nombre “flutter_simple_app”
2. Localizar y abrir el archivo principal “main.dart” en la carpeta “lib”.
3. Modificar el código fuente del archivo “main.dart” siguiendo las siguientes instrucciones:
 - a. Eliminar el código que viene en el archivo.
 - b. Transcribir el código siguiente dentro el archivo “main.dart”:

```
import 'package:flutter/material.dart';
void main() {
  runApp(const MyApp());
}
class MyApp extends StatelessWidget {
  const MyApp({super.key});
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(
          title: const Text('Flutter Simple App'),
        ),
        body: const Center(
          child: Text('¡Hola, mundo Flutter!'),
        ),
      ),
    );
  }
}
```

4. Ejecutar la aplicación:
 - a. Asegurarse de tener un emulador abierto o un dispositivo conectado.
 - b. Ejecutar la aplicación (presionar F5).
5. Ver Resultados:
 - a. Observar cómo la aplicación se lanza en el emulador o dispositivo físico.
 - b. El resultado que se debe ver un mensaje "¡Hola, mundo Flutter!" en la pantalla.

Explicación del código

A continuación, se explica el código de manera detallada.

```
import 'package:flutter/material.dart';
```


Esta línea importa el paquete *material.dart* de Flutter, este paquete contiene widgets y otras utilidades que siguen las directrices de diseño de Material Design, que son ampliamente utilizadas en el desarrollo de aplicaciones Flutter.

```
void main() {  
  runApp(const MyApp());  
}
```

La función *main()* es el punto de entrada para todo programa en Dart. En Flutter, esta función debe llamar a *runApp()* que toma un widget y lo hace el root del árbol de widgets.

MyApp() es una instancia de la clase *MyApp*, que se define a continuación en el código, esta clase será el punto de partida para la aplicación Flutter.

```
class MyApp extends StatelessWidget {  
  const MyApp({super.key});  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      ...  
    );  
  }  
}
```

MyApp es una clase que extiende *StatelessWidget*. En Flutter, los widgets son los bloques básicos para construir la interfaz de usuario y un *StatelessWidget* es un widget que no requiere un estado mutable; es decir, no cambia después de ser construido.

El método *build* es responsable de describir cómo se debe pintar este widget en la UI cada vez que se necesita dibujar (o redibujar) el widget.

BuildContext context es un argumento que se pasa al método *build*, representa la ubicación del widget en el árbol de widgets y puede ser usado para obtener datos del árbol.

```
return MaterialApp(  
  home: Scaffold(  
    ...  
  ),  
);
```

MaterialApp es un widget que envuelve varios widgets que son comúnmente requeridos para aplicaciones de Material Design, proporciona funcionalidad como navegación, temas, etc.

La propiedad *home* de *MaterialApp* define lo que se muestra en la pantalla principal de la aplicación.

Scaffold es un widget que implementa la estructura visual básica de una pantalla de aplicación material, proporciona funcionalidades como una barra de app, un drawer, un snackbar, etc.

```
appBar: AppBar(  
  title: const Text('Flutter Simple App'),  
),
```

La propiedad *appBar* define un *AppBar*, que es una barra de aplicación con un título, *AppBar* es otro widget proporcionado por Material Design que da una barra de herramientas estandarizada en la parte superior de la pantalla.

El *title* del *AppBar* es un widget *Text*, que simplemente muestra texto en la pantalla.

```
body: const Center(  
  child: Text('¡Hola, mundo Flutter!'),  
),
```

La propiedad *body* es parte del *Scaffold* que define el contenido principal de la pantalla, el widget *Center* centra su widget hijo (en este caso, un widget *Text*) en la pantalla, el widget *Text('¡Hola, mundo Flutter!')* muestra el texto "¡Hola, mundo Flutter!".

Cada parte del código contribuye a la construcción de una interfaz de usuario, utilizando el framework de Flutter basado en widgets.

Para concluir este capítulo, se debe hacer notar que la belleza de Flutter reside en cómo componer y anidar widgets para crear interfaces de usuario complejas y personalizadas.